

Phase 2: Blocking & Candidate Generation (Enhanced Plan)

1. The Challenge

Source 1 has ~2.2M records, and Source 2 & 3 have ~5M records each. A naive pair-wise comparison would require 22 Trillion comparisons per source pair, which is computationally impossible.

Blocking reduces this space by generating a small set of highly probable candidate pairs (aiming for <50 pairs per entity) while maintaining >98% recall.

2. Mandatory Pre-Filtering

Country Partitioning: We will perfectly partition the datasets by country (US, India, France).

- Compare US entities in S1 only to US entities in S2/S3.
- Compare India entities in S1 only to India entities in S2/S3.

3. Revised Blocking Pipeline

We will use a multi-strategy union pipeline to maximize recall while using a secondary refinement step to aggressively filter false positives and keep the reduction ratio high.

Step 1: Multi-Key Blocking (Union)

If a pair is generated by ANY of these keys, it becomes a candidate.

- Key A: Improved Exact Match
 - Combine zip_pin + a variable-length name prefix (3-5 chars depending on name length) + the first token of addr_tokens. This handles common prefixes (like "the") and prevents collisions.
- Key B: Double Metaphone (Phonetic)
 - Use jellyfish.metaphone on the first major word of name_key. We will use both the primary and secondary metaphone codes, appended with the state abbreviation, to capture spelling errors and ambiguous pronunciations.
- Key C: Address Token N-Grams
 - Exact match on overlapping 3-gram token sequences of the address to capture variations like "fremont street" vs "fremont st".
- Key D: Sorted Neighborhood Blocking (SNB)

- Sort records by a composite key (e.g., name_key + zip_pin) and slide a window of size $w=20$. This captures near-matches that differ by a single character.
- Key E: MinHash + LSH
 - Use datasketch to generate Locality Sensitive Hashing (LSH) blocks based on token sets (name + address tokens). It provides sub-linear candidate generation for high-dimensional sparse data.

Step 2: TF-IDF Soft Blocking

This catches typos and missing words using `sparse_dot_topn` for blazing-fast sparse matrix multiplication.

- Name Vectorization: Character 2-grams and 3-grams combined. (2-grams for short typos, 3-grams for fragments).
- Address Vectorization: Word 1-grams and 2-grams combined.
- Top-K Retrieval: Retrieve the Top 25-30 matches for names (threshold ~ 0.35) and Top 15 matches for addresses (threshold ~ 0.40).

Step 3: Secondary Similarity Refinement

A cheap pre-filter to drop obviously bad candidates from the union set before expensive Phase 3 feature engineering.

- Jaccard Token Overlap: Calculate overlap of name_key and addr_tokens.
- Drop pairs where the combined weighted overlap is below a loose threshold (e.g., < 0.25).

4. Validation Framework

Before scaling to 12.5M records, we will build a robust evaluation script on a 50,000-row S1 validation split that tracks: 1. Pair Completeness (Recall): Target $>98\%$ true matches found. 2. Reduction Ratio (RR): Target $>99.9\%$ (1000:1). 3. Mean Candidate Set Size: Average candidates per S1 entity. Target <50 . 4. Per-Strategy Analysis: Measure the unique recall contribution of each key (A-E and TF-IDF) and disable redundant strategies.

5. Required Libraries

- `sparse_dot_topn` (Already installed)
- `scikit-learn` (Already installed)
- `jellyfish` (For Double Metaphone)
- `datasketch` (For MinHash + LSH)

6. Output

The output of this phase is `candidate_pairs.tsv` containing:

- `source1_entity_id`

- `target_entity_id`
- `blocking_source` (comma-separated list of strategies that found the pair)